



DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de Software

Análisis y Diseño de Software - NRC 30724

T1U2

Patrones de diseño

Grupo

Saraguro Leidy

Ñacato Alexander

Jaguaco Jonathan

Obando Erick

Instructor: Ing. Jenny Alexandra Ruiz Robalino

Fecha: 21/05/2026

ADAPTER

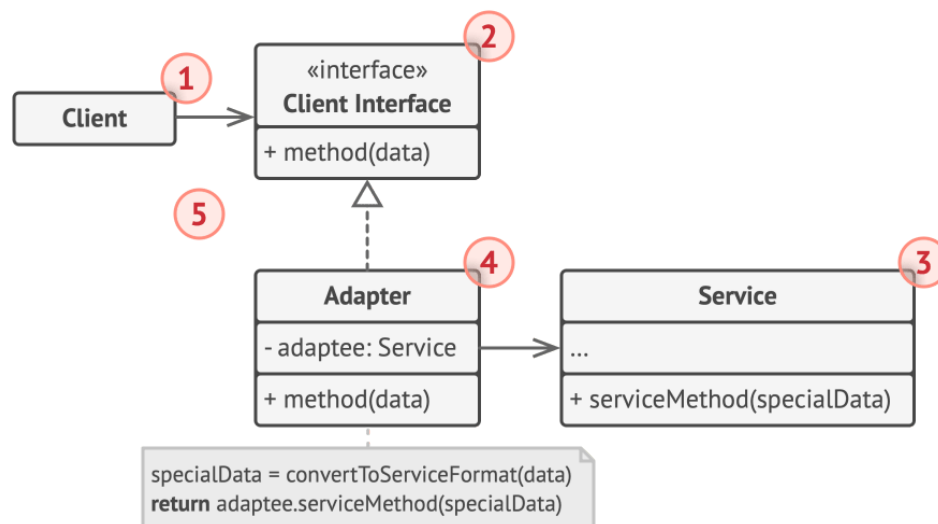
Descripción

Este patrón actúa como un **intermediario o traductor** entre un componente existente (a menudo un componente antiguo o de terceros) y un sistema nuevo. Su función es similar a la de un adaptador físico: así como un adaptador de corriente permite conectar un enchufe de tres clavijas en una toma de dos, el patrón Adapter crea una **abstracción** que mapea la interfaz antigua a la nueva arquitectura..

Características

- **Adaptación posterior al diseño:** A diferencia de otros patrones como Bridge, que se diseñan de antemano, el Adapter se utiliza para hacer que clases no relacionadas funcionen juntas después de haber sido diseñadas.
- **Mecanismo de "Wrapper":** El adaptador envuelve al objeto original (el "Adaptado") para proporcionar una vista traducida de sus métodos..
- **Participantes:** Su estructura se compone sustancialmente de un Cliente , una interfaz Objetivo (la que el cliente necesita), el Adaptado (la clase que requiere ser adaptada) y el Adaptador (la clase que realiza el mapeo entre el Objetivo y el Adaptado).
- **Implementación:** Puede realizarse mediante herencia o mediante agregación (composición).
- **Diferenciación:** Se distingue de otros patrones porque cambia la interfaz del objeto , mientras que un *Proxy* mantiene la misma interfaz y un *Decorator* la mejora sin alterarla.

Estructura



1. La clase **Cliente** contiene la lógica de negocio existente del programa.
2. La **Interfaz con el Cliente** describe un protocolo que otras clases deben seguir para poder colaborar con el código cliente.
3. **Servicio** es alguna clase útil (normalmente de una tercera parte o heredada). El cliente no puede utilizar directamente esta clase porque tiene una interfaz incompatible.
4. La clase **Adaptadora** es capaz de trabajar tanto con la clase cliente como con la clase de servicio: implementa la interfaz con el cliente, mientras envuelve el objeto de la clase de servicio. La clase adaptadora recibe llamadas del cliente a través de la interfaz



TALLER 2

UNIDAD 1

Departamento: Ciencias de la computación

Carrera: Ingeniería de software

Asignatura: Análisis y Diseño de software

NRC: 30724

Apellidos y nombres del estudiante:

Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

de cliente y las traduce en llamadas al objeto envuelto de la clase de servicio, pero en un formato que pueda comprender.

CRUD ESTUDIANTE

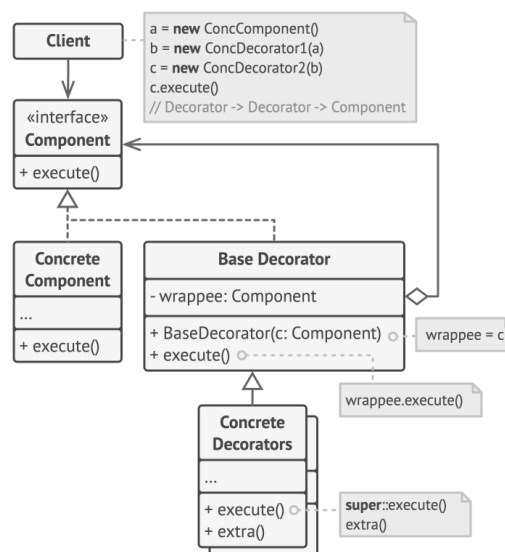
- **Estructura de Mediación (El Adaptador):** Se implementa una clase intermedia que se sitúa entre el flujo de control del sistema y el componente externo.
- **Client (ControlEstudiante):** Constituye el componente que inicia la ejecución del flujo de negocio. Este elemento interactúa exclusivamente con la interfaz objetivo, realizando llamadas de manera agnóstica sin tener conocimiento de las transformaciones internas ni de la existencia de sistemas externos incompatibles.
- **Target (IRepositorioEstudiante):** Define el contrato o la interfaz que el cliente (Client) comprende y utiliza directamente. Establece los métodos estandarizados necesarios para el ciclo de vida del CRUD del estudiante, sirviendo como la frontera de abstracción del sistema.
- **Adapter (EstudianteRepositorioAdapter):** Representa la clase adaptadora que implementa formalmente la interfaz Target. Mantiene una relación de asociación o composición (- adaptee) con el servicio externo, lo que le permite recibir las peticiones estandarizadas, procesarlas, realizar la conversión de tipos o estructuras y delegar el trabajo final al objeto incompatible.
- **Adaptee (ServicioWebESPE):** Corresponde al servicio externo o heredado que posee la lógica requerida o la base de datos centralizada de la institución. Este componente cuenta con una interfaz y firmas de métodos incompatibles, los cuales no pueden ser modificados directamente por el sistema.

DECORATOR

El patrón Decorator es un patrón de diseño estructural que permite añadir funcionalidades a objetos colocando estos objetos dentro de objetos encapsuladores especiales que contienen estas funcionalidades. También es conocido por el sobrenombre de "Wrapper" (envoltorio), ya que la idea principal es vincular un objeto "envoltorio" con un objeto "objetivo" que contiene el comportamiento básico. Este envoltorio implementa la misma interfaz que el objeto envuelto, lo que permite que el cliente los trate como idénticos, mientras el decorador altera el resultado realizando acciones antes o después de pasar la solicitud al objetivo.

Características

- **Dinámico y Flexible:** A diferencia de la herencia, que es estática y no permite alterar la funcionalidad de un objeto en tiempo de ejecución, el Decorator permite añadir responsabilidades de forma dinámica.
- **Composición Recursiva:** Se basa en la composición para organizar un número indefinido de objetos. Esto permite envolver un objeto en múltiples capas de decoradores, logrando un comportamiento combinado.
- **Interfaz Común:** Tanto los componentes básicos como los decoradores siguen una interfaz común, lo que garantiza que sean intercambiables en el código cliente.
- **Delegación de Tareas:** El decorador base contiene un campo para referenciar el objeto envuelto y le delega todas las operaciones, permitiendo que los decoradores concretos solo sobrescriban lo necesario.
- **Estructura de "Pila":** Los objetos resultantes se estructuran como una pila, donde el último decorador añadido es con el que el cliente interactúa directamente.



Implementación del Decorador CRUD estudiante

Definiendo una interfaz **IEstudiante** para tu clase **Estudiante**, y creando una clase decoradora que, al envolver al objeto original, permita que el **ControlEstudiante** añada comportamientos extra sin modificar la lógica base de guardado o eliminación del **RepositorioEstudiante**. De esta forma, el CRUD sigue funcionando igual, pero los objetos "estudiante" que fluyen por el sistema pueden ganar capacidades adicionales.



TALLER 2

UNIDAD 1

Departamento: Ciencias de la computación

Carrera: Ingeniería de software

Asignatura: Análisis y Diseño de software

NRC: 30724

Apellidos y nombres del estudiante:

Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

FACADE

El patrón de diseño **FACADE** sirve para proporcionar una interfaz simplificada de todo el proyecto, encapsulando toda la lógica del sistema en funcionalidades fáciles de entender, donde solo se usa lo necesario del código sin preocuparse por toda la complejidad interna.

Ejemplo de videoconvertidor:

- VideoFile
- OggCompressionCodec
- MPEG4CompressionCodec
- CodecFactory
- VideoConverter

La clase VideoConverter es la fachada porque simplifica todo el proceso de conversión y el usuario solo usa esa clase sin interactuar con todas las demás.

Este patrón fachada se usa:

- Cuando se necesita una interfaz limitada o directa hacia un subsistema complejo.
- Cuando se desea estructurar subsistemas en capas.
- Cuando se quiere ocultar la complejidad del sistema dejando solo funciones fáciles de usar.

Estructura

La estructura del patrón de diseño **Facade** (Fachada) se basa en la creación de un punto de entrada único que simplifica la interacción con un sistema complejo. Sus componentes principales son los siguientes:

1. Fachada (Facade)

Es la clase principal que proporciona un acceso práctico a una parte específica de la funcionalidad del subsistema.

- **Responsabilidad:** Sabe a qué objetos del subsistema debe dirigir la petición del cliente y cómo operar todas las "partes móviles" para obtener un resultado.
- **Intermediación:** Actúa como un abogado o facilitador para el cliente, capturando la complejidad de las colaboraciones internas.



TALLER 2

UNIDAD 1

Departamento: Ciencias de la computación

Carrera: Ingeniería de software

Asignatura: Análisis y Diseño de software

NRC: 30724

Apellidos y nombres del estudiante:

Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

2. Fachadas Adicionales (Optional Additional Facades)

En casos donde una sola fachada se vuelve demasiado grande o "todopoderosa" (objeto dios), se pueden extraer comportamientos a fachadas refinadas o adicionales.

- Esto evita contaminar una única clase con funciones no relacionadas y permite que otras fachadas o clientes las utilicen.

3. Subsistema Complejo (Complex Subsystem)

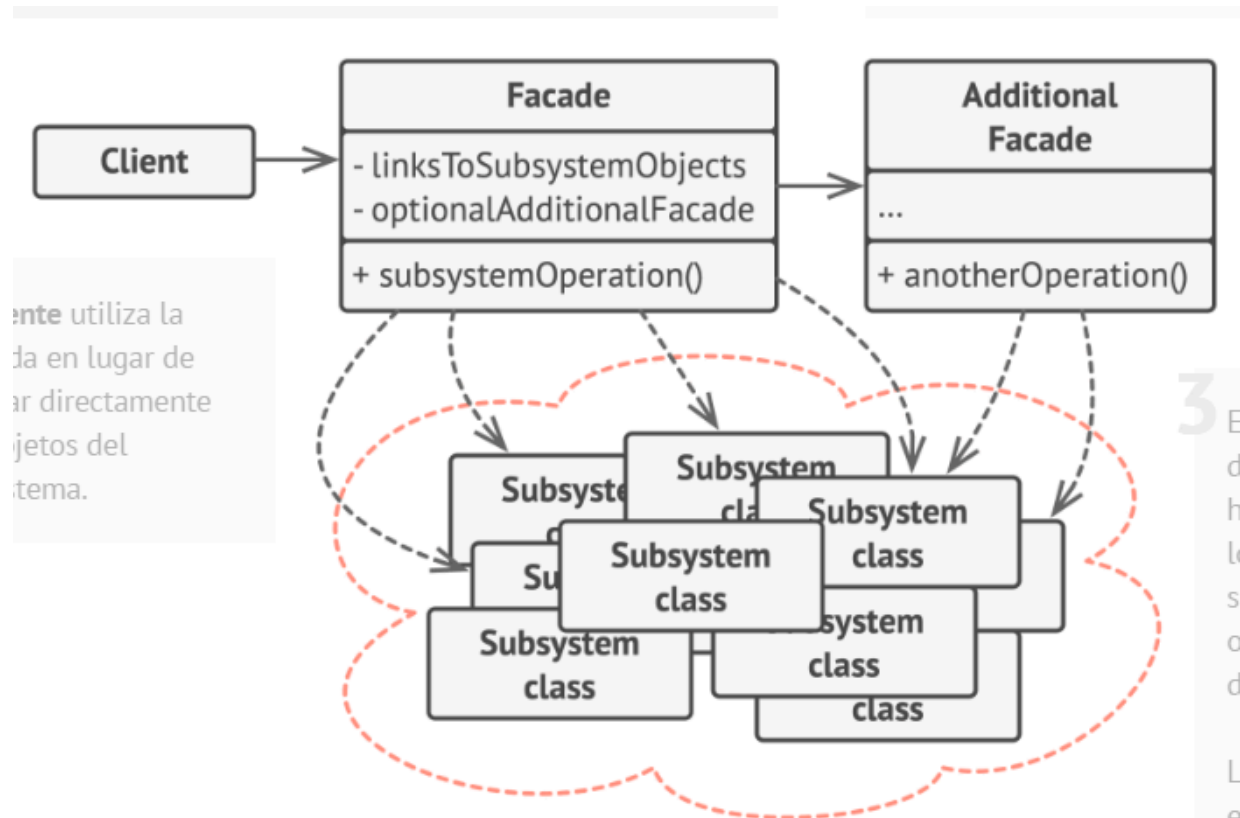
Consiste en decenas de objetos diversos que implementan la lógica de negocio detallada.

- **Independencia:** Las clases del subsistema no conocen la existencia de la fachada; operan directamente entre sí y dentro de su propio contexto.
- **Funcionamiento:** Para que el subsistema haga algo útil, normalmente se requiere inicializar objetos en el orden correcto y suministrar datos en formatos específicos, procesos que la Fachada oculta al cliente.

4. Cliente (Client)

Es el código que necesita utilizar el subsistema.

- **Interacción:** En lugar de invocar directamente a los objetos del subsistema, el cliente **utiliza únicamente la interfaz de la fachada**.
- **Beneficio:** Esto desacopla al cliente de los detalles de implementación, permitiendo que el subsistema evolucione o se sustituya con un esfuerzo mínimo en el código del cliente



Características

- **Interfaz unificada y simplificada:** Proporciona un único punto de acceso sencillo para un conjunto de interfaces en un subsistema complejo.
- **Ocultamiento de complejidad:** Envuelve las "partes móviles" y detalles técnicos del subsistema, mostrando solo lo que es importante para el cliente.
- **Desacoplamiento:** Reduce las dependencias entre los clientes y las clases de terceros o subsistemas, mejorando la portabilidad.
- **Unidireccionalidad de conocimiento:** La fachada conoce las clases del subsistema, pero las clases del subsistema no conocen la existencia de la fachada.
- **Rol de facilitador:** Debe ser un abogado o mediador simple y evitar convertirse en un "objeto todopoderoso" o "clase gorda".
- **Estructuración en capas:** Funciona como un punto de entrada para cada nivel de un sistema, facilitando la comunicación entre capas.
- **Acceso flexible:** No prohíbe que los usuarios avanzados accedan directamente a las clases del sistema si necesitan funciones específicas



TALLER 2

UNIDAD 1

Departamento: Ciencias de la computación

Carrera: Ingeniería de software

Asignatura: Análisis y Diseño de software

NRC: 30724

Apellidos y nombres del estudiante:

Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

FACADE en el CRUD de estudiantes

En el caso del CRUD de estudiantes nos sirvió para la abstracción de algunos componentes del sistema para que sea más simple entender la complejidad del sistema

Tabla Comparativa de Ventajas y Desventajas de los Patrones de Diseño

Adapter, Decorator y Facade

Patrón de Diseño	Ventajas Clave (Pros)	Desventajas Clave (Contras)
Adapter (Adaptador)	1. Intercambiabilidad Eficiente: Permite cambiar módulos de terceros fácilmente usando el Principio Abierto/Cerrado sin romper el código central. 2. Responsabilidad Única: Aísla por completo la lógica de traducción de datos de la lógica de negocio principal. 3. Reutilización Absoluta: Evita escribir o desarrollar módulos complejos desde cero cuando ya existe una solución funcional.	1. Complejidad Inicial: Eleva la cantidad de clases e interfaces intermedias en el ecosistema del proyecto. 2. Mapeos Imperfectos (Leaky Abstractions): Obliga a usar trucos o retornar valores nulos si la tecnología externa no soporta funciones nativas esperadas. 3. Sobrecarga de Rendimiento (Overhead): Introduce una ligera latencia de procesamiento al transformar los datos en tiempo de ejecución.
	1. Adiós a la Explosión de Clases: Ofrece una alternativa dinámica a la herencia sin necesidad de crear subrazas de clases estáticas. 2. Responsabilidad Única Atómica:	1. Depuración Compleja (Muñeca Rusa): Encontrar la raíz de un error se vuelve difícil debido al gran flujo de objetos pequeños anidados.



TALLER 2

UNIDAD 1

Departamento: Ciencias de la computación

Carrera: Ingeniería de software

Asignatura: Análisis y Diseño de software

NRC: 30724

Apellidos y nombres del estudiante:

Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

Decorator (Decorador)

Divide una clase monolítica gigante en pequeños decoradores que manejan comportamientos únicos.

3. Flexibilidad en Tiempo de Ejecución: Permite adjuntar o remover superpoderes y accesorios al objeto sin detener la aplicación.

2. Sensibilidad al Orden de Capas: Modificar accidentalmente el orden de envoltura puede arruinar por completo el comportamiento esperado.

3. Pérdida de Identidad de Objeto: Los wrappers pueden camuflar la clase original, rompiendo los chequeos estrictos de tipo de objeto.

Facade (Fachada)

1. Desacoplamiento de Subsistemas: Protege al cliente final (como interfaces gráficas) de depender directamente de múltiples librerías complejas.

2. Alta Facilidad de Uso (API Amigable): Simplifica interacciones robustas reduciéndolas a una sola línea de código limpia.

3. Orquestación Organizada: Actúa como un recepcionista eficiente que coordina subrutinas en el orden exacto requerido.

1. Riesgo de Objeto Dios (God Object): Si no se limita, la fachada absorberá demasiadas responsabilidades y se acoplará a todo el sistema.

2. Bloqueo de Funciones Avanzadas: Oculta la flexibilidad de configuraciones quirúrgicas, forzando al usuario a saltarse la fachada en casos especiales.

3. Mantenimiento Centralizado Crítico: Cualquier cambio interno profundo en el flujo exige una reestructuración de la Fachada para evitar caídas generales.



TALLER 2

UNIDAD 1

Departamento: Ciencias de la computación

Carrera: Ingeniería de software

Asignatura: Análisis y Diseño de software

NRC: 30724

Apellidos y nombres del estudiante:

Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

Implementación de los patrones de diseño en el CRUD de estudiante

1. Patrón Decorator

@startuml Decorator_CRUD_Estudiante

skinparam classAttributeIconSize 0

interface ServicioEstudiante {

+ registrarEstudiante(id, nombre, edad) : String

+ actualizarEstudiante(id, nombre, edad) : String

+ borrarEstudiante(id) : String

+ obtenerTodos() : List

}

class ControlEstudiante {

- repositorio : RepositorioEstudiante

+ registrarEstudiante(id, nombre, edad) : String

+ actualizarEstudiante(id, nombre, edad) : String

+ borrarEstudiante(id) : String

+ obtenerTodos() : List

+ validarDatos(id, nombre, edad) : boolean

}

abstract class DecoratorEstudiante {

servicio : ServicioEstudiante

+ registrarEstudiante(id, nombre, edad) : String

+ actualizarEstudiante(id, nombre, edad) : String

+ borrarEstudiante(id) : String



TALLER 2

UNIDAD 1

Departamento: Ciencias de la computación

Carrera: Ingeniería de software

Asignatura: Análisis y Diseño de software

NRC: 30724

Apellidos y nombres del estudiante:

Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

```
+ obtenerTodos() : List
}

class DecoratorValidacionMayorEdad {
    - esMayorDeEdad(edad) : boolean
    + registrarEstudiante(id, nombre, edad) : String
    + actualizarEstudiante(id, nombre, edad) : String
}

class DecoratorValidacion {
    - validarDatos(id, nombre, edad) : boolean
    + registrarEstudiante(id, nombre, edad) : String
    + actualizarEstudiante(id, nombre, edad) : String
}

class DecoratorNotificacion {
    + registrarEstudiante(id, nombre, edad) : String
    + actualizarEstudiante(id, nombre, edad) : String
    + borrarEstudiante(id) : String
}

class FrmEstudiante {
    - control : ServicioEstudiante
}

ServicioEstudiante    <|.. ControlEstudiante
ServicioEstudiante    <|.. DecoratorEstudiante
DecoratorEstudiante    <|-- DecoratorValidacionMayorEdad
```



TALLER 2

UNIDAD 1

Departamento: Ciencias de la computación

Carrera: Ingeniería de software

Asignatura: Análisis y Diseño de software

NRC: 30724

Apellidos y nombres del estudiante:

Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

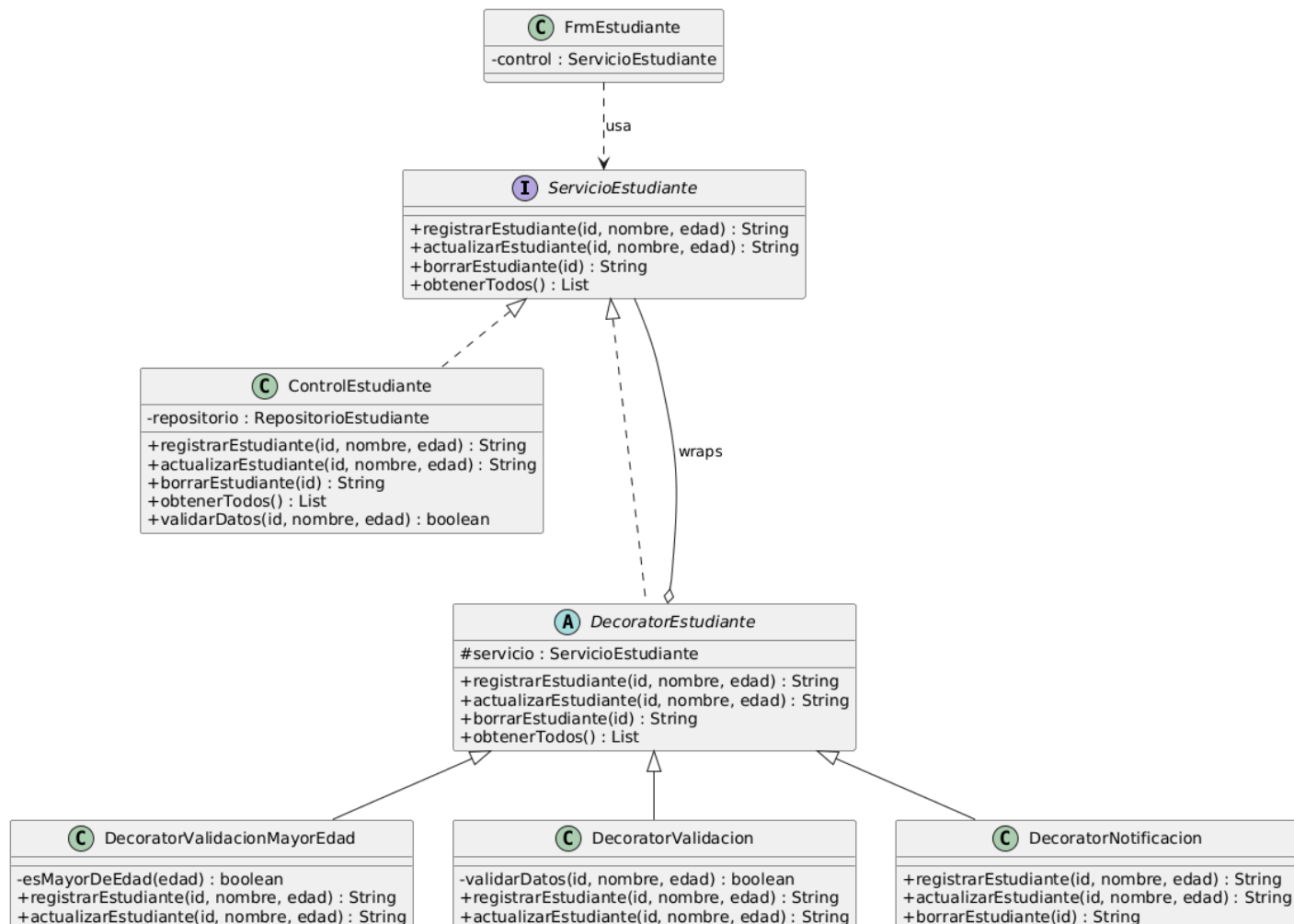
DecoratorEstudiante <|-- DecoratorValidacion

DecoratorEstudiante <|-- DecoratorNotificacion

DecoratorEstudiante o--- ServicioEstudiante : wraps

FrmEstudiante ..> ServicioEstudiante : usa

@enduml





TALLER 2

UNIDAD 1

Departamento: Ciencias de la computación

Carrera: Ingeniería de software

Asignatura: Análisis y Diseño de software

NRC: 30724

Apellidos y nombres del estudiante:

Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

Cómo se implementó el Decorator en el proyecto:

ServicioEstudiante es la interfaz común que permite que todos los decorators sean intercambiables con ControlEstudiante. DecoratorEstudiante es la clase abstracta base que tiene un campo servicio (el objeto envuelto) y delega todas las operaciones hacia él. Los tres decorators concretos solo sobrescriben los métodos que les interesan:

- DecoratorValidacionMayorEdad — corta el flujo si edad < 18
- DecoratorValidacion — valida que ID, nombre y edad sean coherentes
- DecoratorNotificacion — muestra un JOptionPane con el resultado final

Se apilan en FrmEstudiante como capas de cebolla: la llamada entra por DecoratorNotificacion, pasa a DecoratorValidacion, luego a DecoratorValidacionMayorEdad, y finalmente llega al ControlEstudiante real.

2. Patrón Facade

```
@startuml Facade_CRUD_Estudiante
```

```
skinparam classAttributeIconSize 0
```

```
package "Vista" {
```

```
class FrmEstudiante {
```

```
- estadisticas : FachadaEstadisticas
```

```
- servicioMateria : ServicioMateria
```

```
}
```

```
}
```

```
package "Fachadas" {
```

```
class FachadaEstadisticas {
```

```
- controlador : ControlEstudiante
```

```
+ obtenerTotalEstudiantes() : int
```

```
+ obtenerEdadPromedio() : double
```



TALLER 2

UNIDAD 1

Departamento: Ciencias de la computación

Carrera: Ingeniería de software

Asignatura: Análisis y Diseño de software

NRC: 30724

Apellidos y nombres del estudiante:

Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

+ obtenerEstudianteMayorEdad() : Estudiante

+ obtenerMateriaMasPopular() : Materia

+ obtenerReporteGeneral() : String

}

class ServicioMateria {

- controlador : ControlEstudiante

+ asignarMateria(idEstudiante, materia) : String

+ eliminarMateria(idEstudiante, materia) : String

}

}

package "Subsistema" {

class ControlEstudiante {

+ registrarEstudiante() : String

+ obtenerTodos() : List

}

class RepositorioEstudiante {

- baseDatos : List

+ guardar(e)

+ buscarPorId(id) : Estudiante

+ eliminar(id)

}



TALLER 2

UNIDAD 1

Departamento: Ciencias de la computación

Carrera: Ingeniería de software

Asignatura: Análisis y Diseño de software

NRC: 30724

Apellidos y nombres del estudiante:

Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

```
class Estudiante {
```

```
    - id : int
```

```
    - nombre : String
```

```
    - edad : int
```

```
    - materias : List
```

```
}
```

```
class Materia {
```

```
    - nombre : String
```

```
}
```

```
}
```

```
FrmEstudiante    ..> FachadaEstadisticas : usa
```

```
FrmEstudiante    ..> ServicioMateria : usa
```

```
FachadaEstadisticas --> ControlEstudiante : delega
```

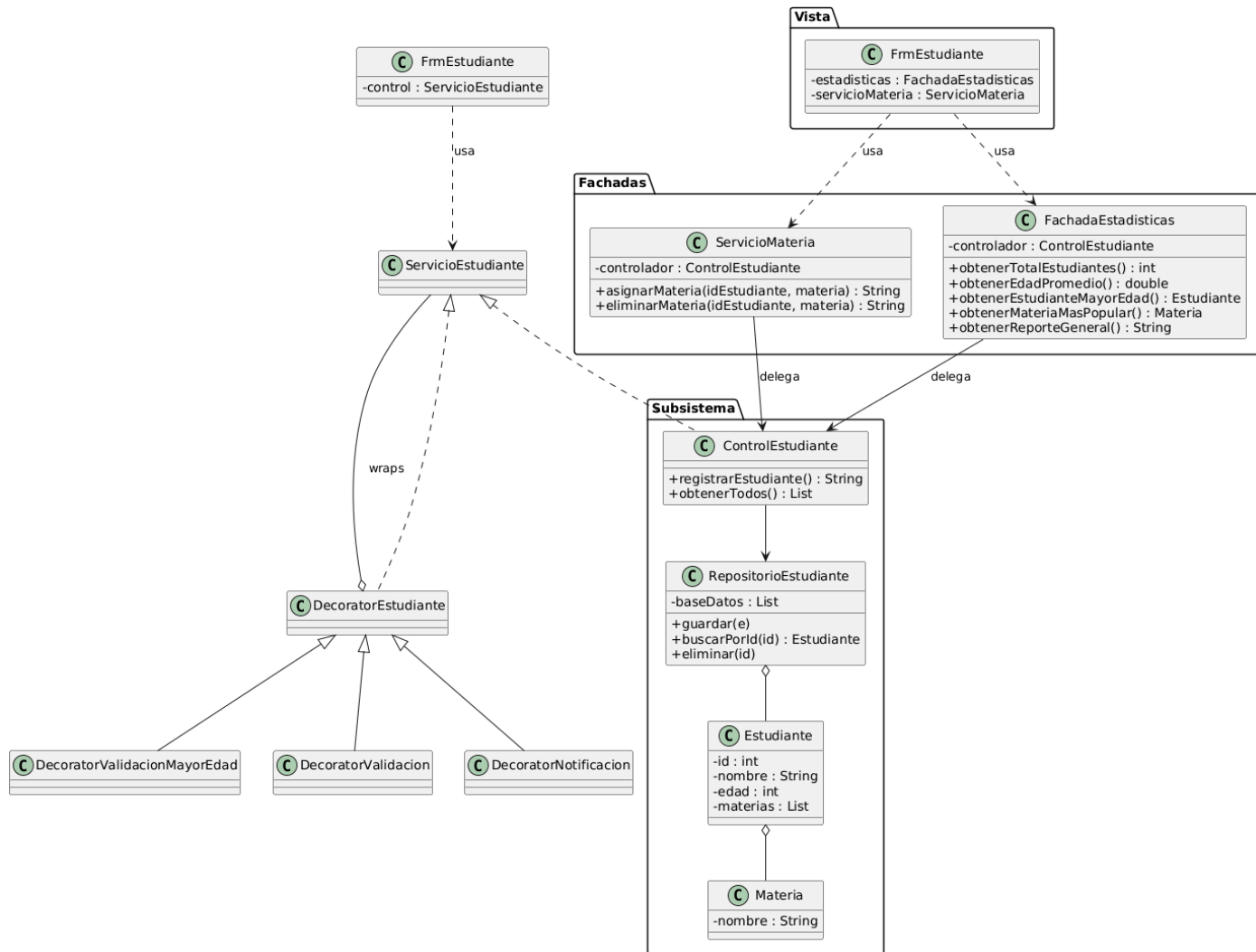
```
ServicioMateria  --> ControlEstudiante : delega
```

```
ControlEstudiante --> RepositorioEstudiante
```

```
RepositorioEstudiante o-- Estudiante
```

```
Estudiante o-- Materia
```

```
@enduml
```



Cómo se implementó el Facade en el proyecto:

La vista FrmEstudiante necesitaba acceder a dos subsistemas complejos: cálculos de estadísticas sobre estudiantes y materias, y la lógica de inscripción/desinscripción de materias. En lugar de que la vista hable directamente con ControlEstudiante, RepositorioEstudiante y Estudiante, se crearon dos fachadas:

- **FachadaEstadisticas**: encapsula 9 métodos de consulta complejos (promedios, máximos, distribuciones, mapas de conteo) y los expone como un único `obtenerReporteGeneral()` que la vista muestra en un `JOptionPane` con una sola línea.



TALLER 2

UNIDAD 1

Departamento: Ciencias de la computación

Carrera: Ingeniería de software

Asignatura: Análisis y Diseño de software

NRC: 30724

Apellidos y nombres del estudiante:

Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

- ServicioMateria: simplifica la lógica de verificar si el estudiante existe, si ya tiene la materia, si es mayor de 18, etc.

El subsistema complejo (ControlEstudiante, RepositorioEstudiante, Estudiante, Materia) no sabe que las fachadas existen, y la vista no sabe nada del subsistema. Eso es exactamente el Facade.

3. Patrón Repository

@startuml Repository_CRUD_Estudiante

skinparam classAttributeIconSize 0

package "Negocio" {

class ControlEstudiante {

- repositorio : RepositorioEstudiante

+ registrarEstudiante(id, nombre, edad) : String

+ actualizarEstudiante(id, nombre, edad) : String

+ borrarEstudiante(id) : String

+ obtenerTodos() : List

}

}

package "Repositorio" {

class RepositorioEstudiante {

- baseDatos : List

+ existeId(id) : boolean

+ guardar(estudiante : Estudiante)

+ buscarPorId(id) : Estudiante

+ actualizar(estudiante : Estudiante)



TALLER 2

UNIDAD 1

Departamento: Ciencias de la computación

Carrera: Ingeniería de software

Asignatura: Análisis y Diseño de software

NRC: 30724

Apellidos y nombres del estudiante:

Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

```
+ eliminar(id)

+ listarTodos() : List

}

}

package "Modelo" {

class Estudiante {

- id : int

- nombre : String

- edad : int

- materias : List

+ agregarMateria(m : Materia)

+ eliminarMateria(m : Materia)

+ tieneMaterias() : boolean

}

class Materia {

- nombre : String

+ equals(obj) : boolean

+ hashCode() : int

+ toString() : String

}

class CatalogoMaterias {

+ {static} obtenerTodasLasMaterias() : List
```



TALLER 2

UNIDAD 1

Departamento: Ciencias de la computación

Carrera: Ingeniería de software

Asignatura: Análisis y Diseño de software

NRC: 30724

Apellidos y nombres del estudiante:

Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

```
+ {static} obtenerMateria(nombre) : Materia  
}  
}
```

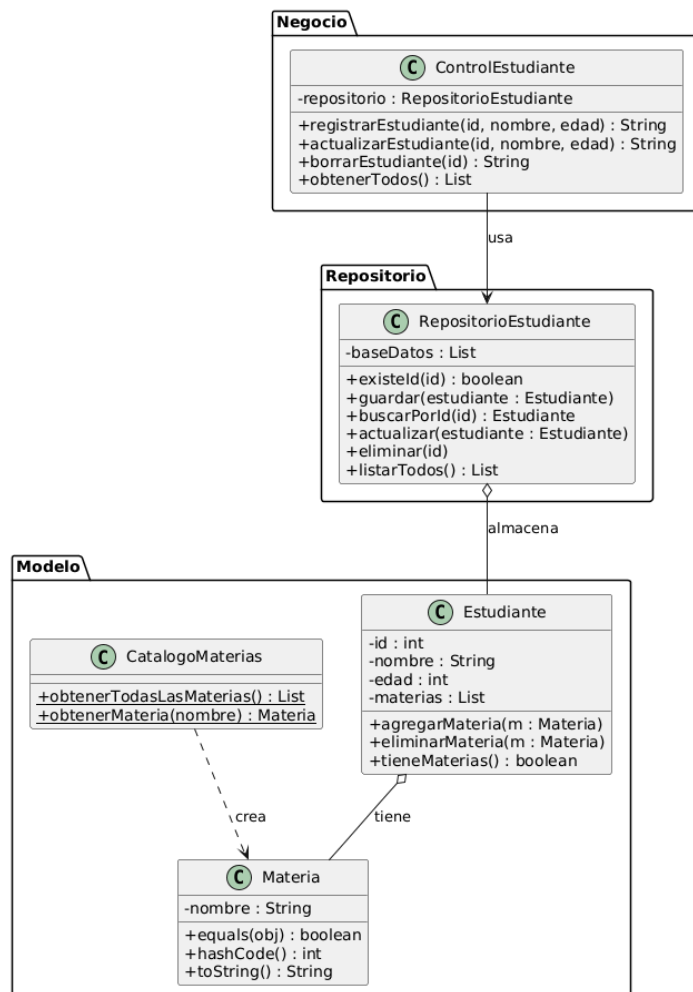
ControlEstudiante --> RepositorioEstudiante : usa

RepositorioEstudiante o-- Estudiante : almacena

Estudiante o-- Materia : tiene

CatalogoMaterias ..> Materia : crea

@enduml





TALLER 2

UNIDAD 1

Departamento: Ciencias de la computación

Carrera: Ingeniería de software

Asignatura: Análisis y Diseño de software

NRC: 30724

Apellidos y nombres del estudiante:

Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

Cómo se implementó el Repository en el proyecto:

RepositoryEstudiante actúa como una "base de datos en memoria" internamente tiene un ArrayList<Estudiante> pero ese detalle está completamente oculto.

ControlEstudiante nunca toca el ArrayList directamente: siempre pide al repositorio con guardar(), buscarPorId(), eliminar(), etc. Si mañana se quisiera cambiar a MySQL o a un archivo JSON, solo se modificaría RepositoryEstudiante y ninguna otra clase se enteraría.

Materia sobrescribe equals() y hashCode() comparando por nombre. Eso es lo que hace que List.contains(materia) y List.remove(materia) en Estudiante funcionen correctamente, porque sin eso Java compararía referencias de objeto y nunca encontraría la materia aunque el nombre sea idéntico.